App Dev Project Report

1. Student Details

Name: Marisetti Likhith Shankar
Roll Number: 22F2000406
Email:22f2000406@ds.study.iitm.ac.in
About Me: I am a student in the IIT Madras BS Degree Program in Data Science and Applications with a strong interest in software engineering, backend systems, and full-stack web development. I enjoy building real-world applications that combine scalable backend architectures, database systems, and interactive user interfaces. My focus is on developing efficient and maintainable web applications using modern technologies such as Flask, Vue.js, and asynchronous task processing frameworks.

---

2. Project Details

Project Title: Udyog – Placement Portal

Problem Statement:

To design and build a web-based placement portal that allows students to apply for job drives posted by companies while enabling companies to manage recruitment processes such as reviewing applications, scheduling interviews, and issuing offer letters.

The platform aims to simulate a **real-world campus placement system** that connects students and companies through a centralized application.

Approach:

The application was built using a **Flask-based backend API and a Vue.js frontend interface**.

The backend handles authentication, database operations, and business logic such as job posting, application processing, interview scheduling, and report generation.

The frontend provides a **dynamic dashboard for students and companies** where they can interact with placement drives, track application status, and manage recruitment processes.

Advanced backend features such as **Celery background tasks and Redis caching** were implemented to automate processes such as sending interview reminders and generating placement reports.

---

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist in writing SQLAlchemy model definitions, creating API documentation samples, and improving variable naming consistency.

The extent of AI/LLM usage is around **10–15%**, limited to **code suggestions and documentation formatting**.

All final implementation logic, debugging, and integration were done manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
Chart.js	To visualize trends such as activity vs. time
Flask-Login	User authentication and session management
WTForms	Frontend form validation
SQLite	Lightweight local database for storing user data
Celery	Background task Processing

---

5. Database Schema / ER Diagram

Tables:

User

- **id**
- **name**
- **email**
- **password**
- **role**
- **phone**
- **is_active**
- **is_approved**

StudentProfile

- **id**
- **user_id**
- **full_name**
- **department**
- **cgpa**
- **skills**
- **resume**

CompanyProfile

- **id**
- **user_id**
- **company_name**
- **industry**
- **website**
- **location**
- **company_size**

Job

- **id**
- **company_id**
- **title**
- **description**
- **skills**
- **salary**

- **job_type**
- **type (JOB / DRIVE)**
- **is_active**

Application

- **id**
- **job_id**
- **student_id**
- **status**
- **interview_date**
- **interview_time**
- **interview_mode**
- **interview_link**
- **feedback**
- **offer_letter**

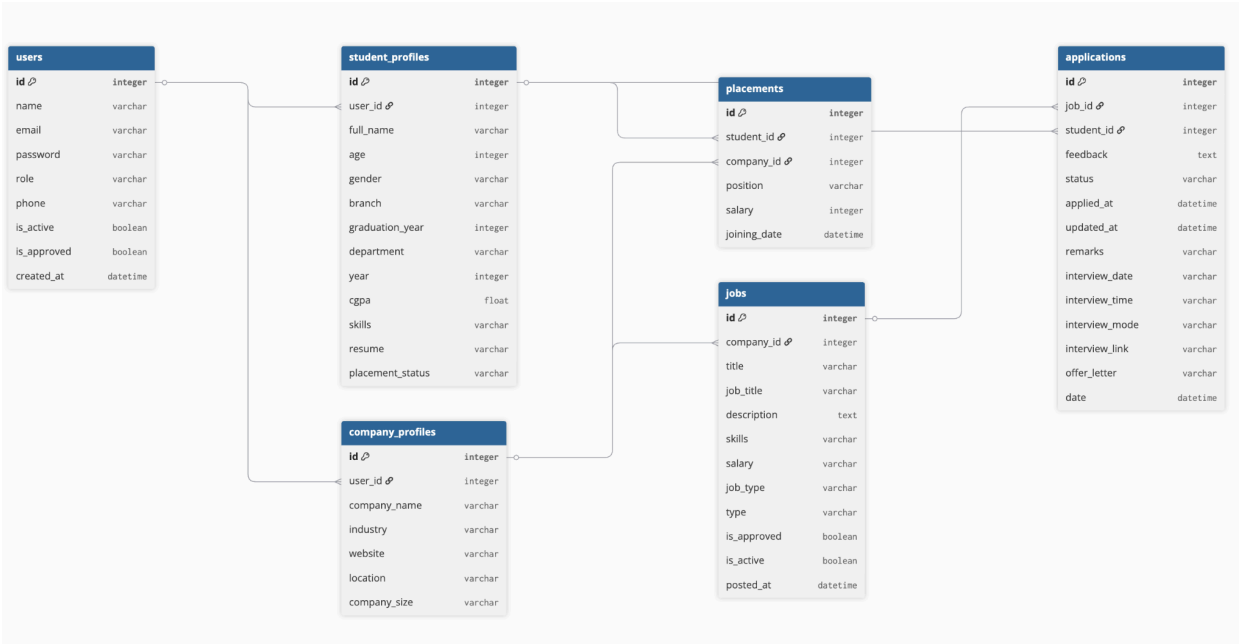
Relationships:

One-to-One

User → StudentProfile
User → CompanyProfile

One-to-Many

CompanyProfile → Job
StudentProfile → Application
Job → Application



---

6. API Resource Endpoints

Endpoint	Method	Description
`/api/login`	POST	Authenticate user and generate session
`/api/activities`	GET	Fetch logged activities for a user
`/api/activity/add`	POST	Add new activity entry
`/api/summary`	GET	Return daily/weekly analytics summary

YAML API Definition File:
Included separately in the submission ZIP as `api.yaml`.

---

7. Architecture and Features (optional)

Architecture Overview:

- **app.py** – main Flask application entry point
- **/models** – database models using SQLAlchemy

- **/routes** – Flask Blueprints for user and activity routes
- **/templates** – Jinja2 HTML templates
- **/static** – CSS, JS, and chart visualization files

Implemented Features:

User registration and login
Daily activity tracking and habit logging
Visualization dashboard using Chart.js
Summary analytics with date filtering
Responsive UI built with Bootstrap
Secure password hashing and user session handling

Additional Features:

- Export user logs as CSV
- AI-generated weekly summary insights (optional extension)

---

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/your-video-link-here
(Accessible to all with "View" permission.)